

Graph Learning using Spectral Sparsity with Heterogeneous Support

Piyush Sawarkar Ganesh Ramakrishnan Ajit Rajwade

piyushs@cse.iitb.ac.in ganesh@cse.iitb.ac.in ajitvr@cse.iitb.ac.in

Department of Computer Science and Engineering
Indian Institute of Technology Bombay, Mumbai, India

Abstract

We present an approach HSGL (Heterogeneous Support Graph Learning), to infer the underlying edge structure of a graph, given a set of nodes and a set of graph signals defined at every node. Most existing techniques for graph learning assume that all the graph signals are bandlimited in the underlying graph Fourier transform (GFT) basis. In a departure from this, HSGL assumes that the underlying signals merely have a sparse support which may extend to the high-frequency basis vectors. Moreover, we also assume that different signals may have different support sets. Under these conditions, we present a three-way alternating minimization strategy to simultaneously infer the graph signal coefficients, the GFT basis vectors, and the eigenvalues of the underlying Laplacian matrix. Furthermore, we also present a strategy to determine the hyper-parameters for a graph learning problem in a principled and purely data-driven fashion. On synthetic data as well as a variety of real-world graphs, we present superior learning results as compared to the state of the art.

1 Introduction

Traditionally, signals or images are defined on uniform Cartesian domains. However, in many different fields, signal values need to be defined at irregularly spaced locations. Typically, a vertex or node of a graph is allocated to each such location, and nodes that are similar in a certain sense are connected by edges. This gives rise to a graph $\mathcal{G} := (\mathcal{V}, \mathcal{E})$ with a node set $\mathcal{V}$ containing n nodes, and edge set $\mathcal{E}$ containing edges of the form (i, j) where $i \in \mathcal{V}, j \in \mathcal{V}, i \neq j$. The edges can be characterized by non-negative weights of the form $w_{ij} \in \mathbb{R}_+$, and these weights may even be binary with $w_{ij} = 0$ and $w_{ij} = 1$ respectively indicating the absence and presence of an edge between nodes i and j . The set of edge weights is also equivalently represented by an adjacency matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$, which is symmetric in case we are dealing with undirected graphs, and which may not be symmetric in case of directed graphs. The set of signal values at the different nodes in $\mathcal{V}$ is called a *graph signal* and is represented in the form of a vector $\mathbf{x} : \mathcal{V} \rightarrow \mathbb{R}^n$. Graph signals arise in diverse fields such as epidemiology (Castellano and Pastor-Satorras 2012), social networks (Tremblay and Borgnat 2014), biological networks (Pirayre et al. 2017), image processing (Cheung et al. 2018; Hu et al. 2014) and computer

graphics (Chen et al. 2017). Associated with the adjacency matrix is also the (combinatorial) Laplacian matrix given by $\mathbf{L} := \mathbf{D} - \mathbf{W}$ where $\mathbf{D}$ is a diagonal matrix of node degrees such that $D_{ii} = \sum_{j=1}^n W_{ij}$. Note that $\mathbf{L} \in \mathbb{R}^{n \times n}$ is also symmetric. Moreover, it is also positive semidefinite which is not necessarily true of $\mathbf{W}$. The eigenvectors of $\mathbf{L}$ are termed the Graph Fourier Transform (GFT) basis vectors and form an orthonormal matrix $\mathbf{V} \in \mathbb{R}^{n \times n}$. In the domain of graph signal processing (GSP), the graph signal $\mathbf{x}$ is expressed as a linear combination of the GFT basis vectors in the form $\mathbf{x} := \mathbf{V}\boldsymbol{\gamma}_{\mathbf{x}}$ where $\boldsymbol{\gamma}_{\mathbf{x}} := \mathbf{V}^\top \mathbf{x}, \boldsymbol{\gamma}_{\mathbf{x}} \in \mathbb{R}^n$ is a vector of GFT coefficients. A wide variety of filtering or sampling operations can be performed on these graph signals with different applications. A comprehensive introduction to and survey of these operations and also of the GSP field is presented in (Ortega et al. 2018).

In many applications, a graph may not be available, and instead needs to be *inferred* from the multivariate signal data defined at the nodes. These signals are defined in the form $\{\mathbf{x}^{(t)}\}_{t=1}^T$ where each signal $\mathbf{x}^{(t)} \in \mathbb{R}^n$. We take this opportunity to define the $n \times T$ coefficient matrix $\boldsymbol{\Gamma}$ such that each column of $\boldsymbol{\Gamma}$ corresponds to the GFT coefficient vector $\boldsymbol{\gamma}_t$ of one of the signals $\mathbf{x}^{(t)}$. Both $\mathcal{V}$ and $\{\mathbf{x}^{(t)}\}_{t=1}^T$ are known, but $\mathcal{E}$ and either $\mathbf{W}$ or $\mathbf{L}$ needs to be inferred. This problem is commonly called *Graph Learning* and has been the subject of extensive research. A comprehensive survey of graph learning techniques is provided in (Dong et al. 2019). In its most basic form, graph learning is an ill-posed problem, which necessitates the use of priors on the graph signals and $\mathbf{W}$ or $\mathbf{L}$ to achieve meaningful solutions. Very commonly, the graph signals are assumed to be smooth or bandlimited *w.r.t.* the underlying (unknown) GFT basis $\mathbf{V}$, that is they are regarded as linear combinations of only the low-frequency GFT basis vectors (Chepuri et al. 2017; Dong et al. 2016; Egilmez et al. 2018; Kalofolias 2016). Other works such as (Batreddy et al. 2023; Humbert et al. 2021; Sardellitti, Barbarossa, and Di Lorenzo 2019) assume a more general model for the graph signals – that they are sparse in the GFT bases, albeit with all signals sharing the same support. This translates to the coefficient matrix $\boldsymbol{\Gamma}$ having only a small subset of rows with non-zero magnitude. This latter assumption is well suited to spectral clustering applications where signals are smooth within a cluster and vary rapidly across clusters.

The assumption of a common support for all signals can be restrictive. This is because the clustering based

Work done as part of Piyush Sawarkar’s MTech Thesis at CSE Department, IIT Bombay, under supervision of Prof. Ajit Rajwade and Prof. Ganesh Ramakrishnan

on one signal (say $\mathbf{x}^{(1)}$) may differ from that based on another signal (say $\mathbf{x}^{(2)}$). In a departure from this, we consider the case where the signals are sparse linear combinations of the GFT basis vectors, with support sets that are completely different from each other. Thus each signal could be the result of a completely different (possibly non-linear) filtering operation on the graph (Ibrahim and Gleich 2019; Mazarguil, Oudre, and Vayatis 2022). We wish to perform graph learning given graph signals in such a setting. We now enlist the contributions of this paper:

1. Given signals $\{\mathbf{x}^{(t)}\}_{t=1}^T$ with $x_i^{(t)}$ denoting the value of the t^{th} signal on the i^{th} node, and having a sparse representation in the GFT basis, we propose a Heterogeneous Support Graph Learning approach, *viz.*, HSGL, to graph learning for inferring the underlying $n \times n$ Laplacian matrix (*c.f.* Section 2). The proposed cost function imposes constraints to ensure that every property of the Laplacian matrix is obeyed.
2. We present a data-driven and principled cross-validation method of parameter tuning (*c.f.* Section 3.2) not just for our technique but for any other technique. This parameter tuning performs cross-validation only on subsets of signals and does not rely on previous cross-validation on any class of graphs, unlike the state-of-the-art.
3. We empirically validate the effectiveness of HSGL on a large number of graphs typically used in network science with synthetically generated signals (*c.f.* Section 3.3). We also present results on real signals in Section 3.4. Our results are superior to those with state-of-the-art techniques in terms of various quality measures.

2 HsGL: Formulation and Algorithm

We now present our algorithm and its mathematical formulation in detail. We begin by stating the assumptions made on the underlying graph $\mathcal{G}$ to be learned from the signal data:

1. We assume that the set of nodes $\mathcal{V}$ is known in advance, and that $|\mathcal{V}| = n$.
2. We assume that $\mathcal{G}$ is undirected and contains no self-loops. We also assume that between any two nodes, there exists at the most one edge.

We make the following assumptions on the available graph signals from which the graph learning task can be accomplished:

1. Each of the T signals has a value defined on each of the n nodes. The signals are represented as $\{\mathbf{x}^{(t)}\}_{t=1}^T$ or equivalently as a matrix $\mathbf{X}$ of size $n \times T$.
2. Each signal $\mathbf{x}^{(t)} = \mathbf{V}\boldsymbol{\gamma}_t$ where $\mathbf{V}$ is the $n \times n$ orthonormal matrix of the eigenvectors of Laplacian matrix $\mathbf{L}$ and $\boldsymbol{\gamma}_t \in \mathbb{R}^n$ is a vector of GFT coefficients. We assume that each $\boldsymbol{\gamma}_t$ is sparse, and the coefficient vectors $\{\boldsymbol{\gamma}_t\}_{t=1}^T$ have different supports. The coefficient vectors can be assembled into a matrix $\boldsymbol{\Gamma}$ of size $n \times T$.

Considering an eigendecomposition of $\mathbf{L}$ of the form $\mathbf{L} = \mathbf{V}\boldsymbol{\Delta}\mathbf{V}^\top$ with eigenvalues in diagonal matrix $\boldsymbol{\Delta}$, we formulate the following cost function to infer $\mathbf{V}, \boldsymbol{\Delta}, \boldsymbol{\Gamma}$ from $\mathbf{X}$ and $\mathcal{V}$:

$$\begin{aligned} \min_{\mathbf{V}, \boldsymbol{\Delta}, \boldsymbol{\Gamma}} & \|\mathbf{X} - \mathbf{V}\boldsymbol{\Gamma}\|_F^2 + \mu_1 \|\boldsymbol{\Gamma}\|_{1,1} + \mu_2 \|\mathbf{V}\boldsymbol{\Delta}\mathbf{V}^\top\|_{1,1} + \\ & \mu_3 \|\hat{\boldsymbol{\Theta}} - \mathbf{V}\boldsymbol{\Delta}\mathbf{V}^\top - \frac{1}{\sigma^2} \mathbf{I}_n\|_F^2 \\ \text{s.t.} & \begin{cases} \mathbf{V}^\top \mathbf{V} = \mathbf{I}_n, & \text{(a)} \\ \frac{1}{\sqrt{n}} \mathbf{1}_n^\top \mathbf{V} = [100 \cdots 0], & \text{(b)} \\ \boldsymbol{\Delta} := \text{diag}(0, \lambda_2, \dots, \lambda_n) \succeq \mathbf{0}, & \text{(c)} \\ \mathbf{V}\boldsymbol{\Delta}\mathbf{V}^\top \mathbf{1}_n = \mathbf{0}_n, & \text{(d)} \\ \text{diag}(\mathbf{V}\boldsymbol{\Delta}\mathbf{V}^\top) \geq \mathbf{0}_n, & \text{(e)} \\ \mathbf{V}\boldsymbol{\Delta}\mathbf{V}^\top - \text{diag}(\text{diag}(\mathbf{V}\boldsymbol{\Delta}\mathbf{V}^\top)) \leq \mathbf{0}_{n \times n}, & \text{(f)} \\ \text{tr}(\boldsymbol{\Delta}) = \text{tr}(\hat{\boldsymbol{\Theta}}) - \frac{n}{\sigma^2}. & \text{(g)} \end{cases} \end{aligned}$$

Here $\mathbf{I}_n$ is an identity matrix of size $n \times n$, $\boldsymbol{\Delta} \succeq \mathbf{0}$ means that $\boldsymbol{\Delta}$ is positive semi-definite, $\mathbf{x} \geq \mathbf{0}_n$ means that each of the n elements of vector $\mathbf{x}$ is greater than or equal to 0, $\mathbf{Q} \leq \mathbf{0}_{n \times n}$ means that every element of the $n \times n$ matrix $\mathbf{Q}$ is less than or equal to 0, $\hat{\boldsymbol{\Theta}}$ is the data precision matrix (or the inverse covariance matrix) calculated as $\hat{\boldsymbol{\Theta}} := (\bar{\mathbf{X}}\bar{\mathbf{X}}^\top / (T-1))^{-1}$, σ denotes the standard deviation of the (zero mean Gaussian) noise in the available signals in $\bar{\mathbf{X}}$ (which is created by deducting the mean vector $\bar{\mathbf{x}}$ from every column of $\mathbf{X}$), and μ_1, μ_2, μ_3 are hyperparameters to weigh different terms of the cost function. There are four terms in the cost function, and they have the following significance: (i) The first term is a data fidelity term that expresses how well the coefficients in $\boldsymbol{\Gamma}$ represent the signals in $\mathbf{X}$ using the eigenvectors in $\mathbf{V}$. (ii) The second term encourages the sparsity of $\boldsymbol{\Gamma}$. (iii) The third term encourages the sparsity of $\mathbf{L}$ (and thereby also of $\mathbf{W}$) using the fact that many practical graphs of interest are sparse in nature. (iv) The aforementioned terms do not impose any fidelity constraint on the eigenvalues in $\boldsymbol{\Delta}$. For this purpose, the fourth term is introduced which uses a fundamental relationship between the data precision matrix and the Laplacian matrix, *i.e.*, $\hat{\boldsymbol{\Theta}} = \mathbf{L} + \mathbf{I}_n/\sigma^2$, following basic principles for Gaussian processes in (Zhu, Lafferty, and Ghahramani 2003, Eqn. 4). This powerful fidelity term has surprisingly been used very sparingly in the literature with (Lake and Tenenbaum 2010) being a notable exception. Additionally, many constraints on $\mathbf{L}, \mathbf{V}, \boldsymbol{\Delta}$ need to be imposed to prevent a trivial solution. Constraint (a) imposes the orthonormality of $\mathbf{V}$, whereas constraint (b) constrains $\mathbf{V}$ to contain an eigenvector with all elements equal to $1/\sqrt{n}$. This eigenvector is associated with eigenvalue 0, since every row of $\mathbf{L}$ sums up to 0, as expressed by constraint (d). Constraint (c) imposes that $\mathbf{L}$ is positive semi-definite with the first eigenvalue being 0. Constraint (e) imposes that the diagonal elements of $\mathbf{L}$ need to be non-negative, whereas constraint (f) imposes that the off-diagonal entries need to be non-positive. Both (e) and (f) are properties of a graph Laplacian matrix. The constraint (g)

further uses the relationship with the trace of the Laplacian which is based on (Lake and Tenenbaum 2010; Zhu, Lafferty, and Ghahramani 2003) and prevents the trivial solution in which all the eigenvalues turn out to be 0.

We refer to our algorithm to minimize the aforementioned cost function as ‘Heterogeneous Support Graph Learning’ or HSGL.

2.1 Optimization

The HSGL optimization problem is not jointly convex in the three variables $\mathbf{\Gamma}$, $\mathbf{\Delta}$ and $\mathbf{V}$.

Algorithm 1: HSGL Algorithm

Input: $\mathbf{X} \in \mathbb{R}^{N \times T}$, σ_{oracle} $\triangleright$ Signals i.e. $\mathbf{X}$: T Observations across n nodes
Output: $\hat{\mathbf{\Gamma}}, \hat{\mathbf{\Delta}}, \hat{\mathbf{V}}$
Initialization:
 $[\mathbf{V}, \mathbf{\Delta}, \mathbf{U}] \leftarrow \text{SVD}(\mathbf{X})$
 $\mathbf{v}_1 \leftarrow \frac{1}{\sqrt{n}} \mathbf{1}_n$ $\triangleright$ Set the first column of $\mathbf{V}$ to be $\frac{1}{\sqrt{n}} \mathbf{1}_n$
 $\mathbf{V} \leftarrow \text{QR}(\mathbf{V})$
 $\bar{\mathbf{x}} \leftarrow \text{mean}(\mathbf{X})$ $\triangleright \bar{\mathbf{x}} \in \mathbb{R}^{n \times 1}$: Mean of observations over each Node
 $\hat{\mathbf{\Theta}} \leftarrow \left(\frac{(\mathbf{X} - \bar{\mathbf{x}})(\mathbf{X} - \bar{\mathbf{x}})^\top}{T-1} \right)^{-1}$ $\triangleright \hat{\mathbf{\Theta}} \in \mathbb{R}^{n \times n}$: Precision Matrix or Inverse Covariance Matrix
 $\sigma = \sigma_{oracle}$
while not converged **do**
 $\triangleright$ $\mathbf{\Gamma}$ -step: Inferring latent signal representation in GFT space:

$$\min_{\mathbf{\Gamma}} \|\mathbf{X} - \mathbf{V}\mathbf{\Gamma}\|_F^2 + \mu_1 \|\mathbf{\Gamma}\|_{1,1}$$

$\triangleright$ $\mathbf{\Delta}$ -step: Inferring eigenvalues of $\mathbf{L}$:

$$\begin{aligned} \min_{\mathbf{\Delta}} \quad & \mu_2 \|\mathbf{V}\mathbf{\Delta}\mathbf{V}^\top\|_{1,1} + \mu_3 \|\hat{\mathbf{\Theta}} - \mathbf{V}\mathbf{\Delta}\mathbf{V}^\top - \frac{1}{\sigma^2} \mathbf{I}_n\|_F^2 \\ \text{s.t.} \quad & \begin{cases} \mathbf{\Delta} := \text{diag}(0, \lambda_2, \dots, \lambda_n) \succeq \mathbf{0}, \\ \mathbf{V}\mathbf{\Delta}\mathbf{V}^\top \mathbf{1}_n = \mathbf{0}_n, \\ \text{diag}(\mathbf{V}\mathbf{\Delta}\mathbf{V}^\top) \geq \mathbf{0}, \\ \mathbf{V}\mathbf{\Delta}\mathbf{V}^\top - \text{diag}(\text{diag}(\mathbf{V}\mathbf{\Delta}\mathbf{V}^\top)) \leq \mathbf{0}, \\ \text{tr}(\mathbf{\Delta}) = \text{tr}(\hat{\mathbf{\Theta}}) - n \frac{1}{\sigma^2}. \end{cases} \end{aligned}$$

$\triangleright$ $\mathbf{V}$ -step: Inferring eigenvectors of $\mathbf{L}$:

$$\begin{aligned} \min_{\mathbf{V}} \quad & \|\mathbf{X} - \mathbf{V}\mathbf{\Gamma}\|_F^2 + \mu_2 \|\mathbf{V}\mathbf{\Delta}\mathbf{V}^\top\|_{1,1} + \\ & \mu_3 \|\hat{\mathbf{\Theta}} - \mathbf{V}\mathbf{\Delta}\mathbf{V}^\top - \frac{1}{\sigma^2} \mathbf{I}_n\|_F^2 \end{aligned}$$

$$\text{s.t.} \quad \begin{cases} \mathbf{V}^\top \mathbf{V} = \mathbf{I}_n, \\ \frac{1}{\sqrt{n}} \mathbf{1}_n^\top \mathbf{V} = [100 \dots 0], \\ \mathbf{V}\mathbf{\Delta}\mathbf{V}^\top \mathbf{1}_n = \mathbf{0}_n, \\ \text{diag}(\mathbf{V}\mathbf{\Delta}\mathbf{V}^\top) \geq \mathbf{0}_n, \\ \mathbf{V}\mathbf{\Delta}\mathbf{V}^\top - \text{diag}(\text{diag}(\mathbf{V}\mathbf{\Delta}\mathbf{V}^\top)) \leq \mathbf{0}_{n \times n}. \end{cases}$$

end while

To solve this problem we alternate between the three variables, minimizing one while keeping the others fixed as shown in Alg. 1. Keeping $\mathbf{V}, \mathbf{\Delta}$ fixed, the function is convex in $\mathbf{\Gamma}$. Keeping $\mathbf{\Gamma}, \mathbf{V}$ fixed, the cost function is convex in $\mathbf{\Delta}$ with convex constraints (c), (e), (f) and (g). Keeping $\mathbf{\Gamma}, \mathbf{\Delta}$ fixed, the cost function itself is convex in $\mathbf{V}$ but the constraints are non-convex. Despite this, we have empirically always observed convergence of the alternating minimization procedure. We have observed speedier results with some loss of accuracy if we implemented a variant which we call HSGL-APPROX and which has the following steps: (i) The matrix $\mathbf{V}$ is set to be equal to the eigenvector matrix of $\hat{\mathbf{\Theta}}$ using the relation $\hat{\mathbf{\Theta}} = \mathbf{L} + \mathbf{I}_n/\sigma^2$, and not further modified. (ii) Only $\mathbf{\Delta}, \mathbf{\Gamma}$ are updated further in an alternating minimization fashion. Our cost function is jointly convex in $\mathbf{\Gamma}$ and $\mathbf{\Delta}$. However, we have still used alternating minimization for efficiency, and the convergence is always guaranteed using biconvexity based results from (Gorski, Pfeuffer, and Klamroth 2007). For our basic HSGL variant, the matrix $\mathbf{V}$ was always initialized using the SVD of signal matrix $\mathbf{X}$ as shown in Alg. 1.

3 Numerical Experiments

3.1 Synthetic Graphs

We performed graph learning experiments for the following several synthetically generated graphs commonly used in the network science community (each with $n = 20$ nodes unless otherwise specified).

1. An Erdos-Renyi (ER) graph model is constructed with an edge creation probability of 0.2.
2. The Barabasi Albert (BA) graph model is used to generate a scale-free random graph by adding one new node to the graph at each time, connecting it to one existing node in the graph, with the probability of the new node being linked to a given existing node in the graph being proportional to the ratio of the degree of that existing node to the sum of degrees of all the existing nodes.
3. A Forest Fire (FF) graph is generated with *forest fire forward/backward burning* probability equal to 0.3.
4. A Stochastic Kronecker (SK) graph model is constructed using an initiator graph of size 3×3 with weights $[0.75, 0.65, 0.55]$. Since we consider undirected graphs in this paper, the directed graphs returned by the above graph model are symmetrized. For this graph, we have $n = 3^3$ nodes.
5. The Watts Strogatz (WS) graph model is used to generate a connected small-world graph. First, create a ring over n nodes. Then each node in the ring is joined to its k nearest neighbors (or $k - 1$ neighbors if k is odd). Then shortcuts are created by replacing some edges as follows: for each edge, (u, v) in the underlying “ n -ring with k nearest neighbors”, with probability p replace it with a new edge (u, w) with a uniformly random choice of existing node w . The

entire process is repeated until a connected graph results. For this graph, we have $n = 20, k = 4, p = 0.1$.

6. The Dorogovtsev, Goltsev & Mendes (DGM) graph model begins with a single edge connecting two nodes at $t = -1$. At each time step, a new node is added to every edge of the graph, attaching to both end nodes of the edge. By $t = 0$, a triangle of edges connects a triple of nodes, and by $t = 1$, the graph consists of 6 nodes connected by 9 edges, and so on. The total number of nodes at “time” t is $N_t = 3(3^t + 1)/2$, and the total number of edges is $L_t = 3^{t+1}$, resulting in an average degree of $k_t = 2L_t/N_t = 4/(1 + 3^{-t})$. For this graph, we have $n = 15$ nodes.
7. The Stochastic Block Model (SBM) with 3 clusters on $n = 20$ nodes with intra-cluster and inter-cluster edge probabilities being 0.5 and 0.1 respectively.

Graph Signal Generation Model: The signal generation model we follow here is inspired from (Humbert et al. 2021), with an important point of distinction (mentioned below). Given a Laplacian matrix $\mathbf{L} = \mathbf{V}\mathbf{\Delta}\mathbf{V}^\top$, we used the following model for generating a graph signal $\mathbf{y} \in \mathbb{R}^N$: $\mathbf{y} = \mathbf{V}\boldsymbol{\gamma} + \boldsymbol{\varepsilon}$, where $\boldsymbol{\varepsilon}$ is a Gaussian noise vector with zero mean and covariance $\sigma^2 \mathbf{I}_n$. To encourage spectral sparsity, we use a distribution that allows $\boldsymbol{\gamma}$ to admit zero-valued components. To this end, we introduce independent latent Bernoulli variables $\{q_i\}_{i=1}^n$ with success probability $\pi_i \in [0, 1]$ for each i . Knowing $\{q_i\}_{i=1}^n$, the conditional distribution for $\boldsymbol{\gamma}$ is given by $\boldsymbol{\gamma} | \mathbf{q} \sim \mathcal{N}(0, \mathbf{\Lambda}^\dagger)$ where $\mathbf{\Lambda}^\dagger$ is the Moore-Penrose pseudo-inverse of the diagonal matrix containing the values $\{\lambda_i \mathbb{I}\{q_i = 1\}\}_{i=1}^n$. In this model, q_i controls the sparsity of the i^{th} element of $\boldsymbol{\gamma}$. Indeed, if $q_i = 0$, then $\gamma_i = 0$ almost surely. On the other hand, if $q_i = 1$, then γ_i follows a Gaussian distribution with zero-mean and variance equal to $1/\lambda_i$. We introduce sparsity by choosing the probabilities $\{\pi_i\}_{i=1}^n$ at random, as opposed to $\{\pi_i\}_{i=1}^n$ values following a power law as in (Humbert et al. 2021) which encourages smoothness or band-limitedness. Note that, since $\lambda_1 = 0, \gamma_1$ follows a centered degenerate Gaussian, *i.e.*, γ_1 is equal to 0 almost surely. Furthermore, if $\pi_i = 1$ for all i , our model reduces to the one proposed by (Dong et al. 2016) which was only focused on the smoothness assumption.

Competing Techniques: Our HSGL technique is pitted against several contemporary and competitive techniques: (i) the technique from (Dong et al. 2016) which is based on signal smoothness; (ii) the technique using row-sparsity of graph signals from (Humbert et al. 2021); (iii) the approach from (Batreddy et al. 2023) which uses a more flexible form of row-sparsity of graph signals than (Humbert et al. 2021), *i.e.*, it allows at most k rows of the coefficient matrix to have more than α non-zero entries; the graphical LASSO (Friedman, Hastie, and Tibshirani 2008) inspired techniques for estimating the precision matrix or the Laplacian matrix in (iv) (Ying et al. 2020) with three different regularizers: MCP, ℓ_1 and SCAD, (v) (Zhao et al. 2019),

(vi) (Nie et al. 2016), (vii) (Egilmez et al. 2017) using ℓ_1 and ℓ_2 regularizers, (viii) (Kumar et al. 2019) and (ix) (Kumar et al. 2020).

Evaluation Measures: Our algorithm is compared to various competing techniques on four measures: (i) Precision $\text{Prec} = \# \text{true edges retrieved} / \# \text{edges retrieved}$, (ii) Recall $\text{Rec} = \# \text{true edges retrieved} / \# \text{true edges}$, (iii) F-measure $\text{FM} = 2 \times \text{precision} \times \text{recall} / (\text{precision} + \text{recall})$, (iv) Correlation coefficient for estimate of Laplacian matrix given by
$$\rho := \frac{\sum_{i,j} (L_{ij} - L_m)(\hat{L}_{ij} - \hat{L}_m)}{\sqrt{\sum_{i,j} (L_{ij} - L_m)^2 \sum_{i,j} (\hat{L}_{ij} - \hat{L}_m)^2}}$$
 where $\hat{\mathbf{L}}$ is

an estimate of the adjacency matrix $\mathbf{L}$, and $L_m, \hat{L}_m$ stand for the mean values of $\mathbf{L}, \hat{\mathbf{L}}$ respectively. All these measures *except* ρ are computed after binarizing the estimate $\hat{\mathbf{W}}$ by setting all $\hat{W}_{ij}$ values below a threshold τ_W to 0 and setting all $\hat{W}_{ij}$ values greater than or equal to τ_W to 1. For any method, the optimal τ_W is selected by cross-validation in a purely data-driven fashion as explained in the next section.

3.2 Cross-Validation Based Hyper-parameter tuning Strategy

The optimal performance of our algorithm will depend on the choice of the hyperparameters $\mu_1, \mu_2, \mu_3, \tau_W$. This is also a common feature of most graph learning algorithms. In the literature, typically an oracular choice of the hyperparameters based on optimal RRMSE or precision/recall has been adopted (e.g.: Sec. 7.2 of (Humbert et al. 2021)). In some cases, the optimal hyperparameter values for some classes of graphs have been recorded and then re-used. This method is not practical because neither the true graph nor even the ‘class’ of the true graph may be known in advance in practical applications. Instead, we adopt a data-driven parameter selection strategy in this paper. We estimate the graph Laplacian matrix using only a randomly chosen subset of some $T' < T$ signals. Let this set be called the reconstruction set S_R and let the remaining set of $T - T'$ signals be called the validation set S_V . Let the estimated Laplacian matrix for a parameter choice $\tilde{\boldsymbol{\mu}} := (\tilde{\mu}_1, \tilde{\mu}_2, \tilde{\mu}_3, \tilde{\tau}_W)$ be denoted by $\hat{\mathbf{L}}_{\tilde{\boldsymbol{\mu}}}$ and let its associated GFT matrix be denoted by $\hat{\mathbf{V}}_{\tilde{\boldsymbol{\mu}}}$. Then the cross-validation error is denoted by

$$\text{CVE}(\tilde{\boldsymbol{\mu}}) := \sum_{t \in S_V} \|\mathbf{x}^{(t)} - \hat{\mathbf{V}}_{\tilde{\boldsymbol{\mu}}} \boldsymbol{\gamma}_{\tilde{\boldsymbol{\mu}}}^{(t)}\|_2^2 \quad (1)$$

where $\boldsymbol{\gamma}_{\tilde{\boldsymbol{\mu}}}^{(t)} := \arg\min_{\boldsymbol{\gamma}} \|\mathbf{x}^{(t)} - \hat{\mathbf{V}}_{\tilde{\boldsymbol{\mu}}} \boldsymbol{\gamma}\|^2 + \tilde{\mu}_1 \|\boldsymbol{\gamma}\|_1$. Our strategy is to determine the parameter set $\tilde{\boldsymbol{\mu}}$ which minimizes $\text{CVE}(\tilde{\boldsymbol{\mu}})$. This is a principled strategy because the validation error is a good data-driven proxy for the (unobservable) mean squared error.

3.3 Discussion for Results on Synthetic Graphs

We consider two signal-generation settings: (i) GFT-bandlimited signals constructed using only low-frequency GFT basis vectors, and (ii) signals generated as random linear combinations of the GFT basis vectors.

Results on GFT-Bandlimited Signals In Tables 1 and 2, we present comparative results between our technique HSGL and various other competing techniques. In this setting, the signals are generated using a linear combination of only *low frequency* graph Fourier transform basis vectors (i.e. they are GFT-bandlimited), computed by eigendecomposition of the underlying graph Laplacian matrix.

Results on Random Linear Combinations of GFT Basis Vectors In Table 3, we present graph learning results for HSGL against all competing methods. These results are present for the case of $T = 1000$ signals for graphs with $n = 20$ nodes. In this setting, the signals were constructed by means of a random (not necessarily bandlimited) linear combination of the GFT vectors. Compare the results of Tables 3 and 4 to those in Tables 1 and 2. For all cases, we observe excellent precision and recall for our technique, as well as the highest F-Measures.

3.4 Results with Real Data

We now present results on graph learning with three different real datasets, popularly used in graph learning.

(1) **US County Temperature Dataset:** This is a collection of temperature records from 45 counties, each with 5844 time steps (Kalnay et al. 1996). The primary goal is to construct a graph that connects counties with similar temperature trends. There are challenges in creating a ground truth graph for this dataset. For example, using closeness of altitude alone or geographical proximity alone is not sufficient, because even nearby regions can have different temperature trends. The study in (Egilmez et al. 2018) indicates that other latent factors play a critical role in the evolution of temperature patterns. This dataset serves as an excellent example to showcase the capability of our method HSGL to balance sparsity without losing smoothness. As shown in Fig. 1, the graph learned by HSGL includes an edge connecting California and Florida, which have similar temperature trends despite lack of geographical proximity. The method in (Egilmez et al. 2018) predominantly focuses on local smoothness, i.e. diffusion from one node to its adjacent nodes. As a result, it fails to capture global relationships like the one between California and Florida. On the other hand, the method from (Dong et al. 2016), which emphasizes global smoothness, successfully captures the link between California and Florida, but is unable to identify many local regional similarities. Our algorithm HSGL excels by integrating both local and global relationships. It captures

the link between California and Florida, as well as other regional similarities, by considering sparsity in both geometric and spectral spaces.

(2) **Animals Dataset:** This dataset from (Kemp and Tenenbaum 2008) comprises 33 animals, each characterized by responses to 102 binary questions (e.g., ‘Lives in water?’ or ‘Can fly?’). For each animal (a node of the graph), the answer to each question is recorded as either 0 or 1, resulting in a feature set of 102 binary signals per animal. The primary goal is to cluster similar animals together based on these responses. The graph learned by HSGL for this dataset is naturally disconnected, reflecting distinct clusters of animals, as shown in Fig. 2. This clustering with HSGL generally appears to be reasonable, aligning well with intuitive groupings of the animals based on their features, in most cases. On the other hand, (Dong et al. 2016) assigns iguana (a reptile) to the same cluster as penguin, ostrich and chicken (all three birds), and alligator (an amphibious reptile) to the same cluster as cat, lion, wolf (terrestrial mammals). (Humbert et al. 2021) assigns penguin (a bird) to the same cluster as trout and salmon (fish species), and penguin and ostrich to different clusters. HSGL did assign rhino to the same cluster as chimp and gorilla, whereas (Nie et al. 2016) assigned elephant to a different cluster as compared to giraffe, deer and camel (all herbivorous terrestrial mammals). The signals in this dataset clearly possess a non-Gaussian distribution, showcasing HSGL’s ability to perform well in model-mismatch settings where traditional Gaussian assumptions do not hold. Also, with only 102 binary signals, this dataset demonstrates HSGL’s effectiveness in learning with a relatively small number of features. Algorithms such as (Kumar et al. 2020; Nie et al. 2016) require a pre-assigned parameter k , representing the number of clusters, within the graph learning framework itself. This is unlike HSGL and (Dong et al. 2016; Humbert et al. 2021; Ortega et al. 2018), where instead k is specified to a spectral clustering algorithm applied on top of the learned graph. Clustering results for the Animals Dataset (Kemp and Tenenbaum 2008) obtained using HSGL as well as different competing methods are presented in Table 6. These results offer a different presentation of those presented in Figure 2.

(3) **US Senate Dataset:** In this dataset obtained from US Senate rollcall record on <https://voteview.com> for the 110th congress (used in (Wai et al. 2019)), each node represents a state from USA. The aim is to predict the ideological leaning of the state (both senators being Republican (R), both senators being Democrats (D), or one senator each being Republican and Democrat (DR)) at the end of the 110th US Congress (Parliamentary) Meeting, based on the State-Senator responses recorded against a total of 657 issues that were raised in the tenure of the entire meeting (2007-09). The aim is predict their ideologies at the end of the two-year term of this US Congress. These prediction results, derived from graphs learned from HSGL, (Dong et al. 2016), (Humbert et al. 2021) and (Nie et al. 2016) followed by

Technique	Prec	Rec	FM	ρ	Prec	Rec	FM	ρ	Prec	Rec	FM	ρ
	SBM	SBM	SBM	SBM	ER	ER	ER	ER	BA	BA	BA	BA
HSGL	1.000	0.978	0.989	0.993	0.950	1.000	0.974	0.990	1.000	1.000	1.000	0.998
(Dong et al. 2016)	0.677	0.978	0.800	0.988	0.760	1.000	0.864	0.981	1.000	0.737	0.848	0.849
(Humbert et al. 2021)	0.518	0.956	0.672	0.957	0.844	1.000	0.916	0.988	1.000	0.895	0.944	0.964
(Batreddy et al. 2023)	0.237	1.000	0.383	0.886	0.187	0.895	0.309	0.857	0.214	0.158	0.182	0.587
(Ying et al. 2020), MCP	0.957	1.000	0.978	0.998	0.691	1.000	0.817	0.998	0.950	1.000	0.974	0.999
(Ying et al. 2020), ℓ_1	1.000	0.956	0.977	0.994	0.974	1.000	0.987	0.998	1.000	0.895	0.944	0.959
(Ying et al. 2020), SCAD	0.957	1.000	0.978	0.998	0.974	1.000	0.987	0.998	0.950	1.000	0.974	0.999
(Zhao et al. 2019)	0.577	1.000	0.732	0.997	0.404	1.000	0.576	0.998	0.413	1.000	0.585	0.999
(Egilmez et al. 2017), CGL R1	0.405	1.000	0.577	0.987	0.389	0.974	0.556	0.980	0.226	1.000	0.369	0.990
(Egilmez et al. 2017), CGL R2	0.395	1.000	0.566	0.987	0.363	0.974	0.529	0.982	0.198	1.000	0.330	0.990
(Nie et al. 2016)	0.824	0.933	0.875	0.955	0.892	0.868	0.880	0.910	0.679	1.000	0.809	0.963
(Kumar et al. 2020), SGL-N	0.978	1.000	0.989	0.997	0.974	1.000	0.987	0.995	0.950	1.000	0.974	0.990
(Kumar et al. 2020), SGL-QP	0.978	1.000	0.989	0.997	0.974	1.000	0.987	0.996	0.976	1.000	0.988	0.994

Technique	Prec	Rec	FM	ρ	Prec	Rec	FM	ρ	Prec	Rec	FM	ρ
	FF	FF	FF	FF	SK	SK	SK	SK	DGM	DGM	DGM	DGM
HSGL	0.972	1.000	0.986	0.993	0.967	0.967	0.967	0.990	1.000	1.000	1.000	0.997
(Dong et al. 2016)	0.971	0.971	0.971	0.964	0.596	0.983	0.742	0.982	0.931	1.000	0.964	0.978
(Humbert et al. 2021)	1.000	0.629	0.772	0.861	0.932	0.917	0.924	0.974	1.000	0.519	0.683	0.809
(Batreddy et al. 2023)	0.197	0.686	0.306	0.818	0.205	0.383	0.267	0.875	0.257	1.000	0.409	0.794
(Ying et al. 2020), MCP	0.972	1.000	0.986	0.999	0.923	1.000	0.960	0.997	0.628	1.000	0.771	0.997
(Ying et al. 2020), ℓ_1	0.972	1.000	0.986	0.999	0.870	1.000	0.930	0.997	0.818	1.000	0.900	0.997
(Ying et al. 2020), SCAD	0.972	1.000	0.986	0.999	0.923	1.000	0.960	0.997	0.628	1.000	0.771	0.997
(Zhao et al. 2019)	0.500	1.000	0.667	0.999	0.423	1.000	0.594	0.996	0.509	1.000	0.675	0.997
(Egilmez et al. 2017), CGL R1	0.340	1.000	0.507	0.990	0.316	0.983	0.478	0.973	0.441	0.963	0.605	0.926
(Egilmez et al. 2017), CGL R2	0.318	1.000	0.483	0.990	0.309	0.983	0.470	0.973	0.424	0.926	0.581	0.930
(Nie et al. 2016)	0.745	1.000	0.854	0.958	0.940	0.783	0.855	0.921	0.692	1.000	0.818	0.985
(Kumar et al. 2020), SGL-N	0.972	1.000	0.986	0.996	0.938	1.000	0.968	0.995	0.964	1.000	0.982	0.995
(Kumar et al. 2020), SGL-QP	0.972	1.000	0.986	0.999	1.000	0.850	0.919	0.969	0.964	1.000	0.982	0.995

Table 1: Results on graph learning from bandlimited signals: Precision, Recall, FM Measure (the harmonic mean of precision and recall) and Correlation Coefficient values ρ (higher is better for all three measures) for Stochastic Block Model (SBM), Erdos-Renyi (ER), Barabasi-Albert (BA), Forest Fire (FF), Stochastic Kronecker (SK) and Dorogovtsev, Goltsev and Mendes (DGM) graphs for different competing techniques for $n = 20, T = 1000$. Compare with Table 1 of the main paper.

Technique	Prec	Rec	FM	ρ
	WS	WS	WS	WS
HSGL	1.000	1.000	1.000	0.997
(Dong et al. 2016)	0.930	1.000	0.964	0.984
(Humbert et al. 2021)	1.000	0.650	0.788	0.872
(Batreddy et al. 2023)	0.211	1.000	0.348	0.908
(Ying et al. 2020), MCP	0.727	1.000	0.842	0.998
(Ying et al. 2020), ℓ_1	0.952	1.000	0.976	0.998
(Ying et al. 2020), SCAD	0.727	1.000	0.842	0.998
(Zhao et al. 2019)	0.500	1.000	0.667	0.998
(Egilmez et al. 2017), CGL R1	0.426	1.000	0.597	0.988
(Egilmez et al. 2017), CGL R2	0.367	1.000	0.537	0.992
(Nie et al. 2016)	0.870	1.000	0.930	0.985
(Kumar et al. 2020), SGL-N	0.930	1.000	0.964	0.994
(Kumar et al. 2020), SGL-QP	0.976	1.000	0.988	0.994

Table 2: Results on graph learning from bandlimited signals: Precision, Recall, FM Measure (the harmonic mean of precision and recall) and Correlation Coefficient values ρ (higher is better for all measures) for Watts-Strogatz (WS) graphs for different competing techniques for $n = 20, T = 1000$. Compare with Table 2 of the main paper.

Technique	Prec	Rec	FM	ρ	Prec	Rec	FM	ρ	Prec	Rec	FM	ρ
	SBM	SBM	SBM	SBM	ER	ER	ER	ER	BA	BA	BA	BA
HSGL	0.978	1.000	0.989	0.995	0.949	0.974	0.961	0.984	0.905	1.000	0.950	0.978
HSGL-APPROX	0.850	0.980	0.910	0.980	0.910	0.820	0.860	0.950	0.950	1.000	0.970	0.980
(Dong et al. 2016)	0.557	0.978	0.710	0.980	0.469	1.000	0.639	0.966	1.000	0.474	0.643	0.719
(Humbert et al. 2021)	0.620	0.978	0.759	0.967	0.821	0.605	0.697	0.850	0.652	0.789	0.714	0.816
(Batreddy et al. 2023)	0.248	0.711	0.368	0.826	0.190	1.000	0.319	0.851	0.235	0.421	0.302	0.783
(Ying et al. 2020), MCP	0.750	1.000	0.857	0.988	0.679	0.947	0.791	0.965	0.905	1.000	0.950	0.995
(Ying et al. 2020), ℓ_1	0.875	0.933	0.903	0.987	0.967	0.763	0.853	0.947	0.864	1.000	0.927	0.998
(Ying et al. 2020), SCAD	0.750	1.000	0.857	0.988	0.679	0.947	0.791	0.965	0.905	1.000	0.950	0.995
(Zhao et al. 2019)	0.358	0.978	0.524	0.896	0.279	0.895	0.425	0.734	0.190	1.000	0.319	0.850
(Egilmez et al. 2017), CGL R1	0.358	0.978	0.524	0.956	0.333	0.947	0.493	0.903	0.171	1.000	0.292	0.938
(Egilmez et al. 2017), CGL R2	0.358	0.978	0.524	0.956	0.311	0.974	0.471	0.910	0.158	1.000	0.273	0.943
(Nie et al. 2016)	0.971	0.733	0.835	0.900	0.766	0.947	0.847	0.930	0.452	1.000	0.623	0.972
(Kumar et al. 2020), SGL-N	1.000	0.822	0.902	0.903	0.970	0.842	0.901	0.965	0.895	0.895	0.895	0.969
(Kumar et al. 2020), SGL-QP	1.000	0.933	0.966	0.986	0.968	0.789	0.870	0.942	0.900	0.947	0.923	0.947

Technique	Prec	Rec	FM	ρ	Prec	Rec	FM	ρ	Prec	Rec	FM	ρ
	FF	FF	FF	FF	SK	SK	SK	SK	DGM	DGM	DGM	DGM
HSGL	0.897	1.000	0.946	0.982	0.833	0.917	0.873	0.955	1.000	1.000	1.000	0.997
HSGL-APPROX	0.940	0.890	0.910	0.960	0.600	0.930	0.730	0.950	0.960	1.000	0.980	0.980
(Dong et al. 2016)	0.254	1.000	0.405	0.896	0.455	1.000	0.625	0.957	0.257	1.000	0.409	0.795
(Humbert et al. 2021)	0.767	0.657	0.708	0.828	0.700	0.700	0.700	0.874	0.422	1.000	0.593	0.967
(Batreddy et al. 2023)	0.175	1.000	0.298	0.820	0.157	0.783	0.261	0.850	0.255	0.519	0.341	0.756
(Ying et al. 2020), MCP	0.814	1.000	0.897	0.969	0.557	0.983	0.711	0.961	0.900	1.000	0.947	0.993
(Ying et al. 2020), ℓ_1	0.789	0.857	0.822	0.957	0.825	0.867	0.846	0.962	0.844	1.000	0.915	0.989
(Ying et al. 2020), SCAD	0.814	1.000	0.897	0.969	0.557	0.983	0.711	0.961	0.900	1.000	0.947	0.993
(Zhao et al. 2019)	0.351	0.943	0.512	0.794	0.700	0.817	0.754	0.802	0.310	1.000	0.474	0.977
(Egilmez et al. 2017), CGL R1	0.285	1.000	0.443	0.941	0.261	0.983	0.413	0.881	0.435	1.000	0.607	0.892
(Egilmez et al. 2017), CGL R2	0.281	0.971	0.436	0.942	0.261	0.983	0.413	0.881	0.435	1.000	0.607	0.892
(Nie et al. 2016)	0.717	0.943	0.815	0.958	0.776	0.867	0.819	0.937	0.711	1.000	0.831	0.982
(Kumar et al. 2020), SGL-N	0.897	1.000	0.946	0.976	0.793	0.767	0.780	0.798	1.000	0.963	0.981	0.981
(Kumar et al. 2020), SGL-QP	1.000	0.829	0.906	0.960	0.700	0.933	0.800	0.960	0.931	1.000	0.964	0.990

Table 3: Precision, Recall, FM Measure (the harmonic mean of precision and recall) and Correlation Coefficient values ρ (higher is better for all three measures) for Stochastic Block Model (SBM), Erdos-Renyi (ER), Barabasi-Albert (BA), Forest Fire (FF), Stochastic Kronecker (SK) and Dorogovtsev, Goltsev and Mendes (DGM) graphs for different competing techniques for $n = 20, T = 1000$.

Technique	Prec	Rec	FM	ρ
	WS	WS	WS	WS
HSGL	1.000	0.975	0.987	0.989
HSGL-APPROX	1.000	0.950	0.970	0.960
(Dong et al. 2016)	1.000	0.400	0.571	0.768
(Humbert et al. 2021)	0.931	0.675	0.783	0.869
(Batreddy et al. 2023)	0.202	0.925	0.332	0.894
(Ying et al. 2020), MCP	0.833	1.000	0.909	0.991
(Ying et al. 2020), ℓ_1	0.929	0.975	0.951	0.990
(Ying et al. 2020), SCAD	0.833	1.000	0.909	0.991
(Zhao et al. 2019)	0.435	1.000	0.606	0.927
(Egilmez et al. 2017), CGL R1	0.345	1.000	0.513	0.970
(Egilmez et al. 2017), CGL R2	0.345	1.000	0.513	0.970
(Nie et al. 2016)	0.870	1.000	0.930	0.977
(Kumar et al. 2020), SGL-N	1.000	0.925	0.961	0.943
(Kumar et al. 2020), SGL-QP	0.816	1.000	0.899	0.991

Table 4: Precision, Recall, FM Measure (the harmonic mean of precision and recall) and Correlation Coefficient values ρ (higher is better for all measures) for Watts-Strogatz (WS) graphs for different competing techniques for $n = 20, T = 1000$.

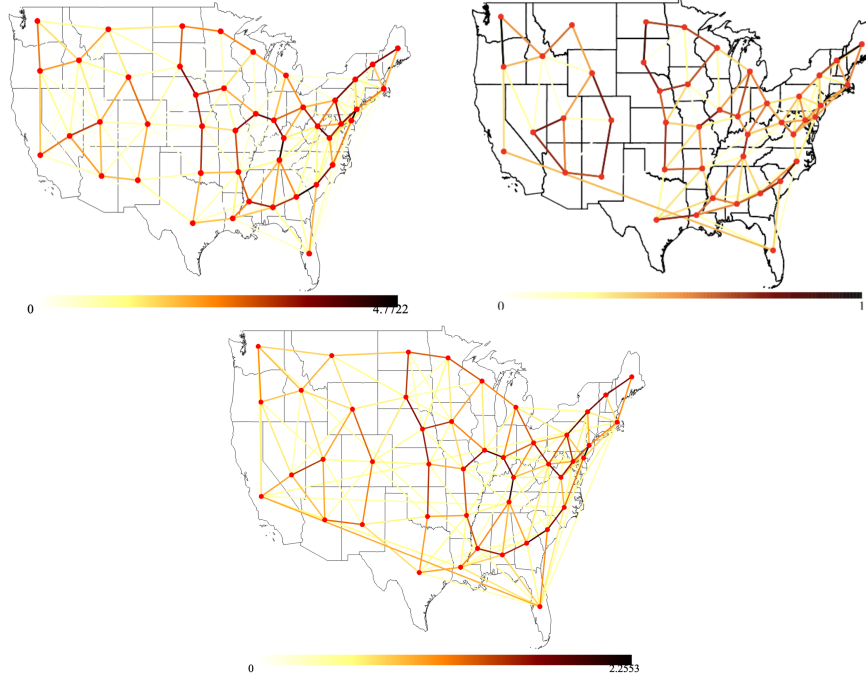


Figure 1: Graphs learned by the methods in (Egilmez et al. 2017) (left), (Dong et al. 2016) (middle) and HSGL (right). HSGL learns the edge between California and Florida which have similar temperature trends despite lack of geographical proximity, as well as local similarities in temperature patterns.



Figure 2: Left to right: Results of graph learning using HSGL, (Dong et al. 2016), (Humbert et al. 2021) and (Nie et al. 2016)

Technique	n_T (/49)	n_D (/17)	n_R (/17)	n_{DR} (/15)
HSGL	33	13	11	9
(Dong et al. 2016)	31	17	0	14
(Humbert et al. 2021)	20	8	6	6
(Nie et al. 2016)	35	4	16	15

Table 5: Results of ideological predictions from graphs learned on the US Senate Dataset using different methods. See text for more details.

Authors →	HSGL	Dong'16	Humbert'21	Nie'16	Zhao'19	Ortega'18	SGL
Cluster 1	Cow,Horse, Elephant, Deer,Giraffe,Camel	Cow,Horse, Rhino,Elephant, Deer,Giraffe, Camel	Cow,Horse, Rhino,Elephant, Deer,Giraffe,Camel	Cow,Horse, Deer,Giraffe,Camel	Cow,Mouse, Horse,Squirrel	Cow,Horse, Mouse	Cow,Horse, Rhino,Elephant, Deer,Giraffe, Camel
Cluster 2	Cat,Lion,Tiger, Wolf,Dog	Cat,Lion,Tiger, Wolf,Dog, Alligator	Cat,Lion,Tiger, Wolf,Dog	Cat,Lion,Tiger, Wolf,Dog	Cat,Lion,Tiger, Wolf,Dog	Cat,Lion,Tiger, Wolf,Dog	Cat,Lion,Tiger, Wolf,Dog
Cluster 3	Dolphin,Seal, Whale,Penguin	Dolphin,Seal, Whale	Dolphin,Seal, Whale	Dolphin,Seal, Whale,Penguin	Dolphin,Seal, Whale	Dolphin,Seal, Whale,Penguin	Dolphin,Seal, Whale
Cluster 4	Trout,Salmon	Trout,Salmon	Trout,Salmon, Penguin	Trout,Salmon	Trout,Salmon, Iguana,Alligator	Trout,Salmon	Trout,Salmon
Cluster 5	Alligator,Iguana	Iguana,Penguin, Chicken,Ostrich	Alligator,Iguana	Alligator,Iguana	Eagle,Robin, Finch	Alligator,Iguana	Alligator
Cluster 6	Chimp,Gorilla, Rhino	Chimp,Gorilla	Chimp,Gorilla	Chimp,Gorilla	Chimp,Gorilla	Chimp,Gorilla, Squirrel	Chimp,Gorilla
Cluster 7	Chicken,Eagle, Robin,Ostrich, Finch	Eagle,Robin, Finch	Chicken,Ostrich	Chicken,Eagle, Robin,Ostrich, Finch	Chicken, Ostrich,Penguin	Chicken,Elephant, Ostrich	Chicken,Eagle, Robin,Penguin, Finch
Cluster 8	Cockroach,Ant	Cockroach,Ant	Cockroach,Ant, Butterfly,Bee	Cockroach,Ant, Butterfly,Bee	Cockroach,Ant	Cockroach,Ant, Butterfly,Bee	Cockroach,Ant, Butterfly,Bee
Cluster 9	Butterfly,Bee	Butterfly,Bee	Eagle,Robin, Finch	Rhino,Elephant	Butterfly,Bee	Eagle,Robin, Finch	Ostrich,Iguana
Cluster 10	Squirrel,Mouse	Squirrel,Mouse	Squirrel,Mouse	Squirrel,Mouse	Camel,Giraffe, Rhino,Deer, Elephant	Camel,Giraffe, Rhino,Deer	Squirrel,Mouse

Table 6: Clustering results for the Animals Dataset (Kemp and Tenenbaum 2008) obtained by means of different methods. Compare with Figure 2 of the main paper.

spectral clustering with $k = 3$ clusters in each case, are presented in Table 5. To represent state opinions during a roll call while building the graph, we sum the votes of the two Senators, with ‘yay’ as 1, ‘nay’ as -1, and ‘absent’ as 0 (same as in (Wai et al. 2019)). Each state is treated as a node in a 50-node graph, producing $L = 657$ graph signals with values 0, 1, 2. Let n_D and n_R respectively be the number of states whose both senators were Democrat (called ‘D-states’) and whose both senators were Republican (called ‘R-states’). Let n_{DR} be the number of states for which one senator each were Republican and Democrat (called ‘DR-states’). Table 5 presents the number of states for which these labels were correctly assigned by each of the algorithms, for each of the three categories (D,R,DR). It also presents the sum total of correct predictions (n_T) across all categories. For example, HSGL correctly labeled 13 out of 17 D-states as democrat, correctly labeled 11 out of 17 R-states as republican, and 9 out of 15 DR-states as mixed (i.e. DR). Hence the total accuracy was 33 out of 49. The total is out of 49 and not 50 because we are ignoring the one state that had one independent senator. As seen in Table 5, our method and (Nie et al. 2016) produce the highest accuracy predictions.

4 Conclusion

We have presented a technique HSGL for graph learning from signals defined on the graph nodes. Our carefully designed cost function allows for graph learning from signals which have sparse representations in the under-

lying GFT basis with potentially completely disparate support sets. The cost function also uses appropriate information available in the covariance matrix of the signal data. We have proposed an alternating minimization procedure for the optimization of the cost function with results on synthetic and real data that surpass those produced by many existing approaches. Although we have numerically observed convergence, we have not established theoretical convergence guarantees. This is an important avenue for future work. Furthermore, extending HSGL to handle non-Gaussian noise, including heavy-tailed noise in signals and missing signal values, is also an important avenue. Lastly, there is also the important problem of graph learning in the signal-starved regime where the number of available signals is less than the number of nodes, where priors on graph Laplacians will play a very important role.

References

- Batreddy, S.; et al. 2023. Robust graph learning for classification. *Signal Processing*, 211: 109120.
- Castellano, C.; and Pastor-Satorras, R. 2012. Competing activation mechanisms in epidemics on networks. *Scientific reports*, 2(1): 371.
- Chen, S.; Tian, D.; Feng, C.; Vetro, A.; and Kovačević, J. 2017. Fast resampling of three-dimensional point clouds via graphs. *IEEE Transactions on Signal Processing*, 66(3): 666–681.
- Chepuri, S. P.; Liu, S.; Leus, G.; and Hero, A. O. 2017.

- Learning sparse graphs under smoothness prior. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 6508–6512. IEEE.
- Cheung, G.; Magli, E.; Tanaka, Y.; and Ng, M. K. 2018. Graph spectral image processing. *Proceedings of the IEEE*, 106(5): 907–930.
- Dong, X.; Thanou, D.; Frossard, P.; and Vandergheynst, P. 2016. Learning Laplacian matrix in smooth graph signal representations. *IEEE Transactions on Signal Processing*, 64(23): 6160–6173.
- Dong, X.; Thanou, D.; Rabbat, M.; and Frossard, P. 2019. Learning graphs from data: A signal representation perspective. *IEEE Signal Processing Magazine*, 36(3): 44–63.
- Egilmez, H. E.; et al. 2017. Graph learning from data under Laplacian and structural constraints. *IEEE Journal of Selected Topics in Signal Processing*, 11(6): 825–841.
- Egilmez, H. E.; et al. 2018. Graph learning from filtered signals: Graph system and diffusion kernel identification. *IEEE Transactions on Signal and Information Processing over Networks*, 5(2): 360–374.
- Friedman, J.; Hastie, T.; and Tibshirani, R. 2008. Sparse inverse covariance estimation with the graphical lasso. *Biostatistics*, 9(3): 432–441.
- Gorski, J.; Pfeuffer, F.; and Klamroth, K. 2007. Biconvex sets and optimization with biconvex functions: a survey and extensions. *Mathematical methods of operations research*, 66(3): 373–407.
- Hu, W.; Cheung, G.; Ortega, A.; and Au, O. C. 2014. Multiresolution graph fourier transform for compression of piecewise smooth images. *IEEE Transactions on Image Processing*, 24(1): 419–433.
- Humbert, P.; Le Bars, B.; Oudre, L.; Kalogeratos, A.; and Vayatis, N. 2021. Learning Laplacian matrix from graph signals with sparse spectral representation. *Journal of Machine Learning Research*, 22(195): 1–47.
- Ibrahim, R.; and Gleich, D. 2019. Nonlinear diffusion for community detection and semi-supervised learning. In *The World Wide Web Conference*, 739–750.
- Kalnay, E. c.; Kanamitsu, M.; Kistler, R.; Collins, W.; Deaven, D.; Gandin, L.; Iredell, M.; Saha, S.; White, G.; Woollen, J.; et al. 1996. The NMC/NCAR 40-year reanalysis project. *Bull. Am. Meteorol. Soc.*, 77(3): 437–472.
- Kalofolias, V. 2016. How to learn a graph from smooth signals. In *Artificial intelligence and statistics*, 920–929. PMLR.
- Kemp, C.; and Tenenbaum, J. B. 2008. The discovery of structural form. *Proceedings of the National Academy of Sciences*, 105(31): 10687–10692.
- Kumar, S.; Ying, J.; Cardoso, J. V. d. M.; and Palomar, D. 2019. Structured graph learning via Laplacian spectral constraints. *Advances in neural information processing systems*, 32.
- Kumar, S.; Ying, J.; Cardoso, J. V. d. M.; and Palomar, D. P. 2020. A unified framework for structured graph learning via spectral constraints. *Journal of Machine Learning Research*, 21(22): 1–60.
- Lake, B.; and Tenenbaum, J. 2010. Discovering structure by learning sparse graphs. In *Proceedings of the Annual Meeting of the Cognitive Science Society*, volume 32.
- Mazarguil, A.; Oudre, L.; and Vayatis, N. 2022. Non-smooth interpolation of graph signals. *Signal processing*, 196: 108480.
- Nie, F.; Wang, X.; Jordan, M.; and Huang, H. 2016. The constrained laplacian rank algorithm for graph-based clustering. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30.
- Ortega, A.; Frossard, P.; Kovačević, J.; Moura, J. M. F.; and Vandergheynst, P. 2018. Graph Signal Processing: Overview, Challenges, and Applications. *Proceedings of the IEEE*, 106(5): 808–828.
- Pirayre, A.; Couprie, C.; Duval, L.; and Pesquet, J.-C. 2017. BRANE Clust: Cluster-assisted gene regulatory network inference refinement. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 15(3): 850–860.
- Sardellitti, S.; Barbarossa, S.; and Di Lorenzo, P. 2019. Graph topology inference based on sparsifying transform learning. *IEEE Transactions on Signal Processing*, 67(7): 1712–1727.
- Tremblay, N.; and Borgnat, P. 2014. Graph wavelets for multiscale community mining. *IEEE Transactions on Signal Processing*, 62(20): 5227–5239.
- Wai, H.-T.; Segarra, S.; Ozdaglar, A. E.; Scaglione, A.; and Jadbabaie, A. 2019. Blind community detection from low-rank excitations of a graph filter. *IEEE Transactions on signal processing*, 68: 436–451.
- Ying, J.; et al. 2020. Nonconvex sparse graph learning under Laplacian constrained graphical model. *Advances in Neural Information Processing Systems*, 33: 7101–7113.
- Zhao, L.; Wang, Y.; Kumar, S.; and Palomar, D. P. 2019. Optimization algorithms for graph Laplacian estimation via ADMM and MM. *IEEE Transactions on Signal Processing*, 67(16): 4231–4244.
- Zhu, X.; Lafferty, J.; and Ghahramani, Z. 2003. *Semi-supervised learning: From Gaussian fields to Gaussian processes*. School of Computer Science, Carnegie Mellon University.